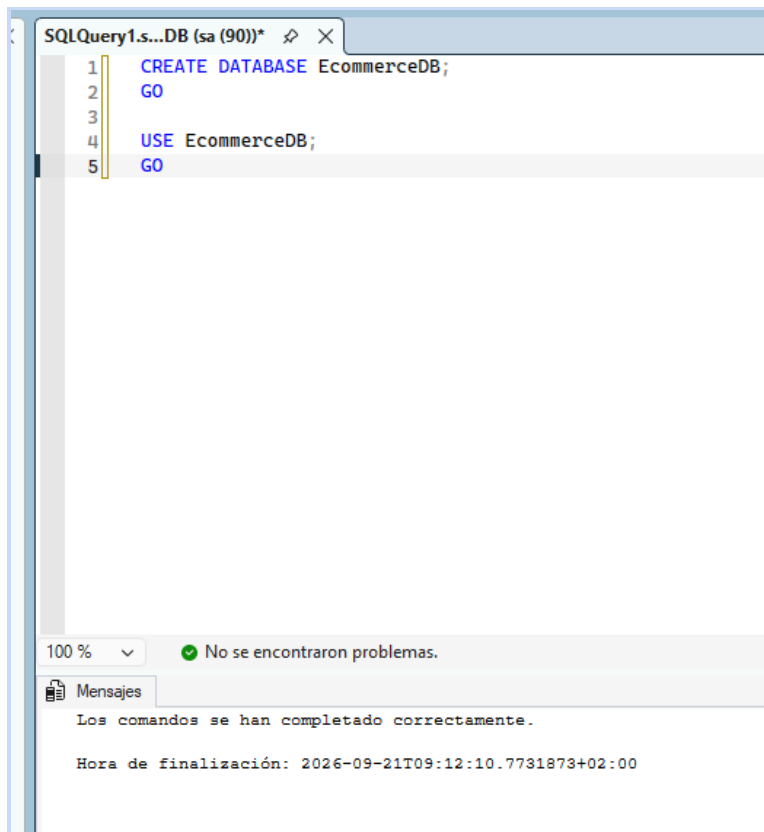


Laboratorio 01 - Create and maintain database objects

- Create a new database

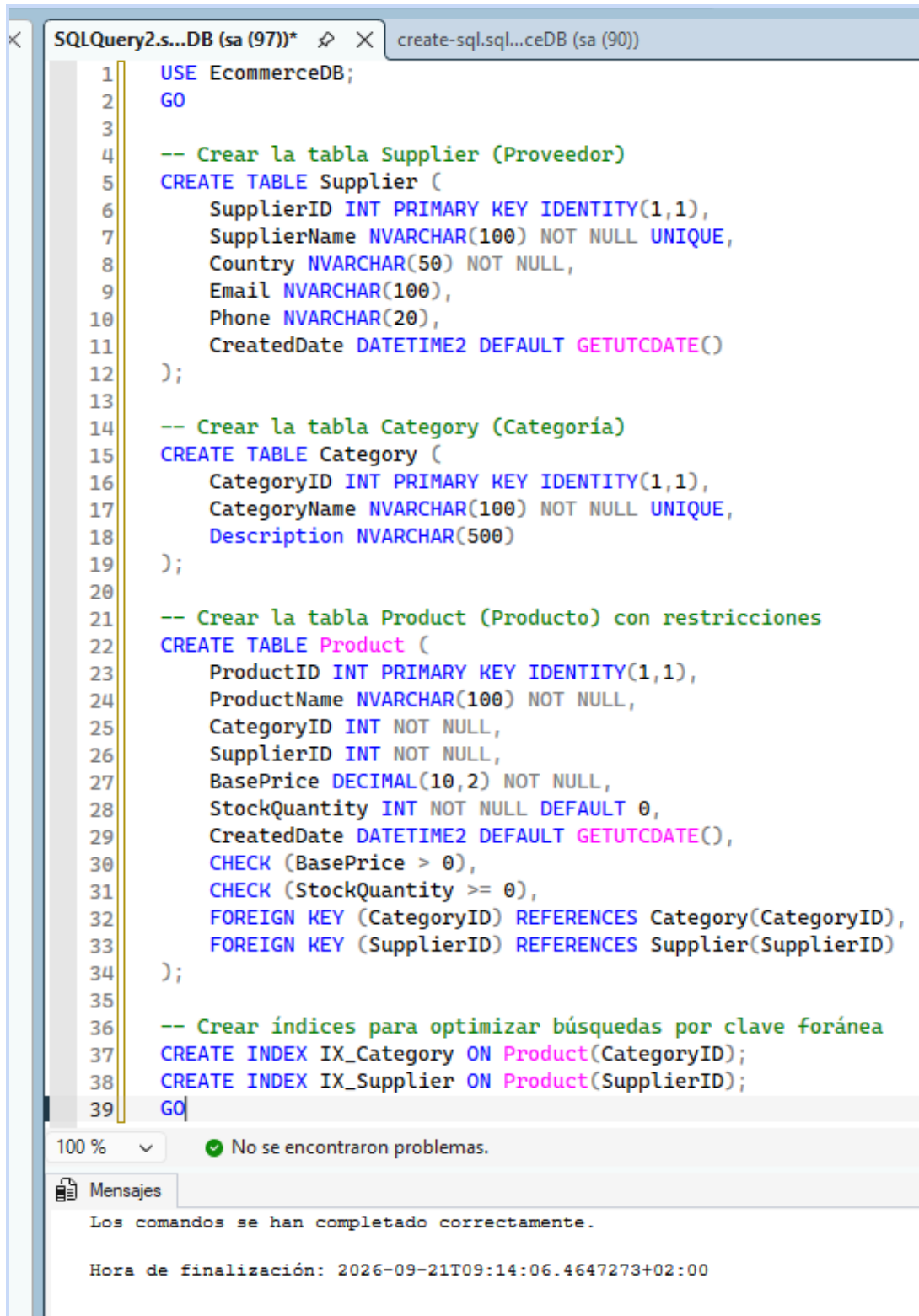
- Creamos la base de datos **EcommerceDB** para empezar a trabajar.



- Create core tables with constraints

- Creamos las tablas principales (Supplier, Category, Product) con reglas de validación e insertamos datos.

1. Crear las tablas e índices.



```
1  USE EcommerceDB;
2  GO
3
4  -- Crear la tabla Supplier (Proveedor)
5  CREATE TABLE Supplier (
6      SupplierID INT PRIMARY KEY IDENTITY(1,1),
7      SupplierName NVARCHAR(100) NOT NULL UNIQUE,
8      Country NVARCHAR(50) NOT NULL,
9      Email NVARCHAR(100),
10     Phone NVARCHAR(20),
11     CreatedDate DATETIME2 DEFAULT GETUTCDATE()
12 );
13
14 -- Crear la tabla Category (Categoría)
15 CREATE TABLE Category (
16     CategoryID INT PRIMARY KEY IDENTITY(1,1),
17     CategoryName NVARCHAR(100) NOT NULL UNIQUE,
18     Description NVARCHAR(500)
19 );
20
21 -- Crear la tabla Product (Producto) con restricciones
22 CREATE TABLE Product (
23     ProductID INT PRIMARY KEY IDENTITY(1,1),
24     ProductName NVARCHAR(100) NOT NULL,
25     CategoryID INT NOT NULL,
26     SupplierID INT NOT NULL,
27     BasePrice DECIMAL(10,2) NOT NULL,
28     StockQuantity INT NOT NULL DEFAULT 0,
29     CreatedDate DATETIME2 DEFAULT GETUTCDATE(),
30     CHECK (BasePrice > 0),
31     CHECK (StockQuantity >= 0),
32     FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),
33     FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)
34 );
35
36 -- Crear índices para optimizar búsquedas por clave foránea
37 CREATE INDEX IX_Category ON Product(CategoryID);
38 CREATE INDEX IX_Supplier ON Product(SupplierID);
39 GO
```

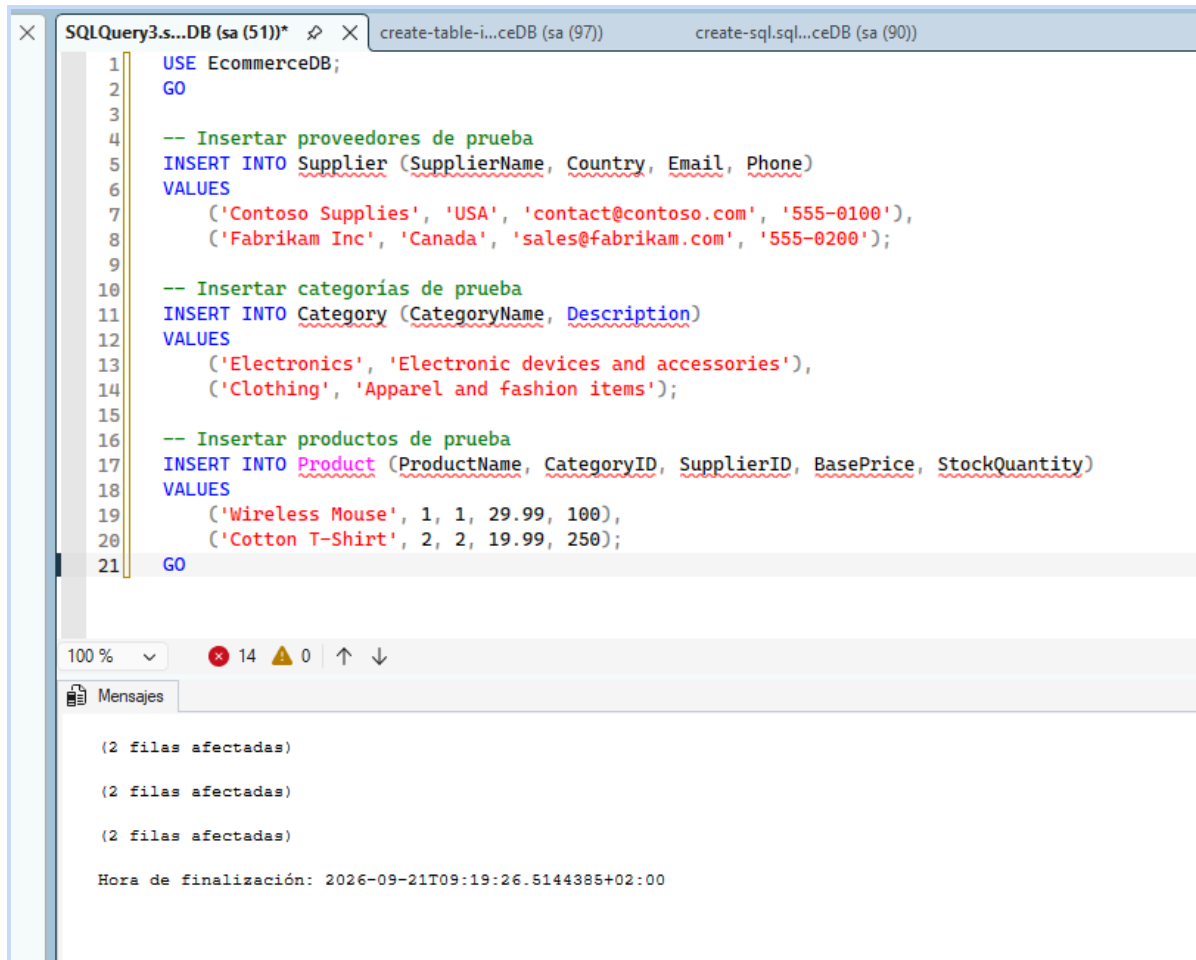
100 %  No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-21T09:14:06.4647273+02:00

2. Insertar datos iniciales.



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a SQL query window titled 'SQLQuery3.s...DB (sa (51))' containing the following SQL script:

```
1 USE EcommerceDB;
2 GO
3
4 -- Insertar proveedores de prueba
5 INSERT INTO Supplier (SupplierName, Country, Email, Phone)
6 VALUES
7     ('Contoso Supplies', 'USA', 'contact@contoso.com', '555-0100'),
8     ('Fabrikam Inc', 'Canada', 'sales@fabrikam.com', '555-0200');
9
10 -- Insertar categorías de prueba
11 INSERT INTO Category (CategoryName, Description)
12 VALUES
13     ('Electronics', 'Electronic devices and accessories'),
14     ('Clothing', 'Apparel and fashion items');
15
16 -- Insertar productos de prueba
17 INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
18 VALUES
19     ('Wireless Mouse', 1, 1, 29.99, 100),
20     ('Cotton T-Shirt', 2, 2, 19.99, 250);
21 GO
```

The bottom pane shows the 'Mensajes' (Messages) window, which displays the execution results of the query:

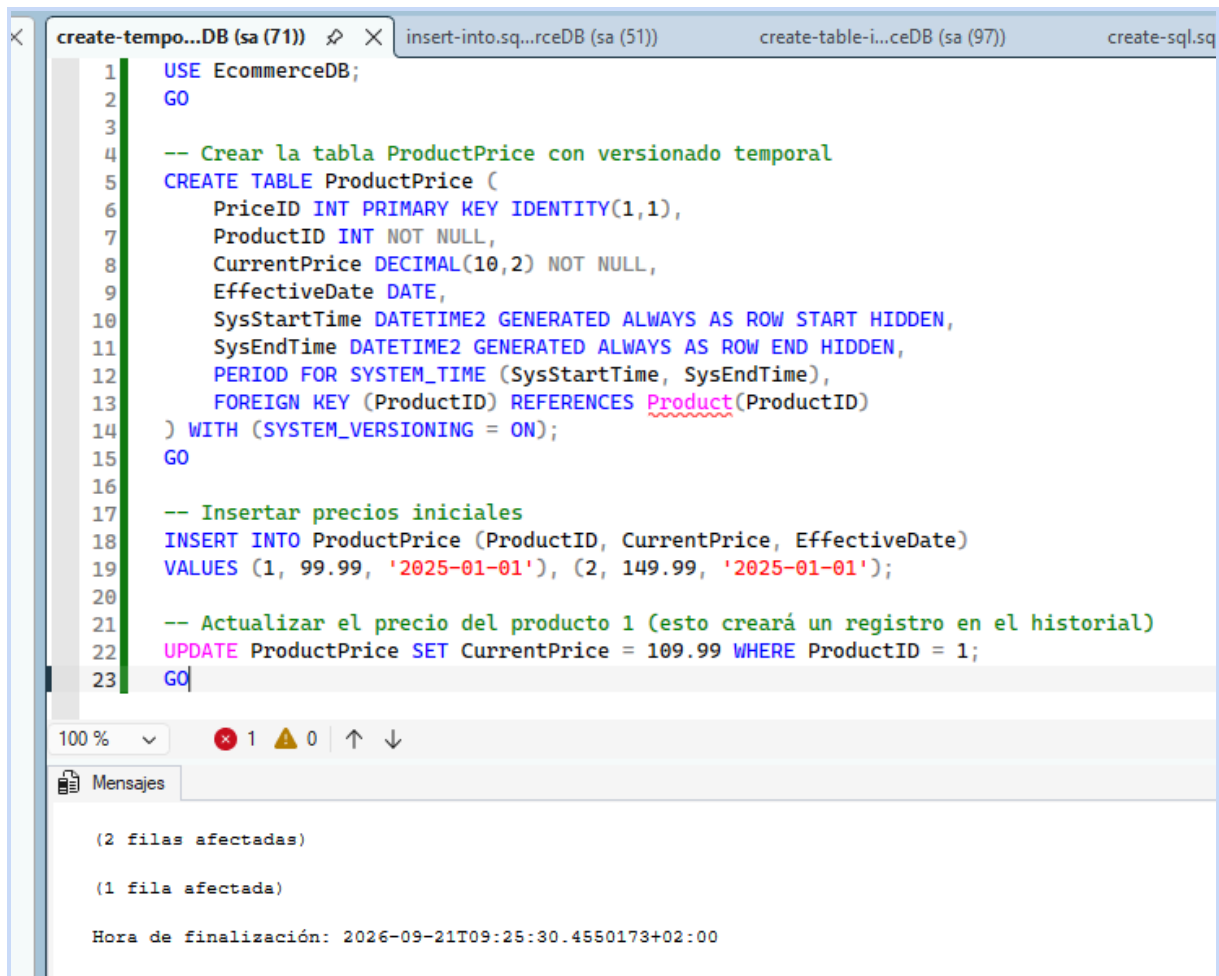
```
(2 filas afectadas)
(2 filas afectadas)
(2 filas afectadas)

Hora de finalización: 2026-09-21T09:19:26.5144385+02:00
```

- **Create a temporal table for price history**

- Creamos la tabla temporal **ProductPrice** para registrar automáticamente el historial de cambios de precios.

1. Crear la tabla temporal e insertar datos



```
1  USE EcommerceDB;
2  GO
3
4  -- Crear la tabla ProductPrice con versionado temporal
5  CREATE TABLE ProductPrice (
6      PriceID INT PRIMARY KEY IDENTITY(1,1),
7      ProductID INT NOT NULL,
8      CurrentPrice DECIMAL(10,2) NOT NULL,
9      EffectiveDate DATE,
10     SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START HIDDEN,
11     SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END HIDDEN,
12     PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),
13     FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
14 ) WITH (SYSTEM_VERSIONING = ON);
15 GO
16
17 -- Insertar precios iniciales
18 INSERT INTO ProductPrice (ProductID, CurrentPrice, EffectiveDate)
19 VALUES (1, 99.99, '2025-01-01'), (2, 149.99, '2025-01-01');
20
21 -- Actualizar el precio del producto 1 (esto creará un registro en el historial)
22 UPDATE ProductPrice SET CurrentPrice = 109.99 WHERE ProductID = 1;
23 GO
```

100 % 1 0

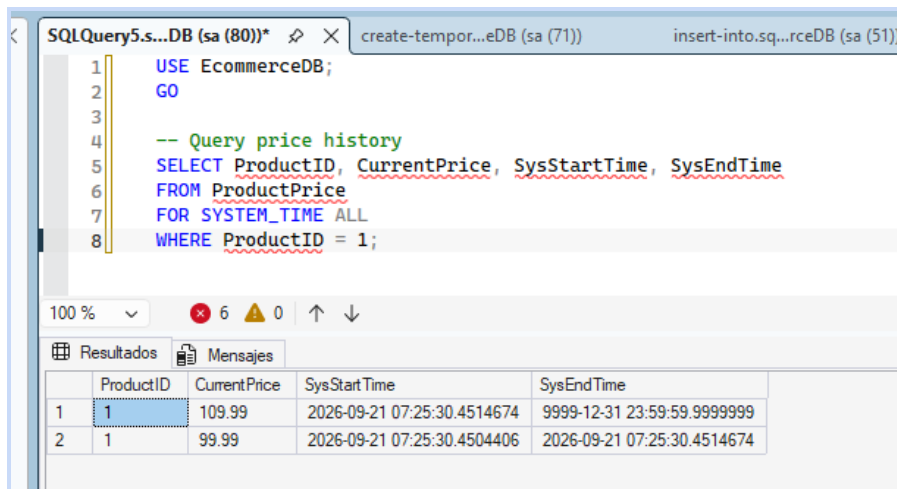
Mensajes

(2 filas afectadas)

(1 fila afectada)

Hora de finalización: 2026-09-21T09:25:30.4550173+02:00

2. Consultar el historial de precios



The screenshot shows a SQL query window with the following code:

```
1 USE EcommerceDB;
2 GO
3
4 -- Query price history
5 SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
6 FROM ProductPrice
7 FOR SYSTEM_TIME ALL
8 WHERE ProductID = 1;
```

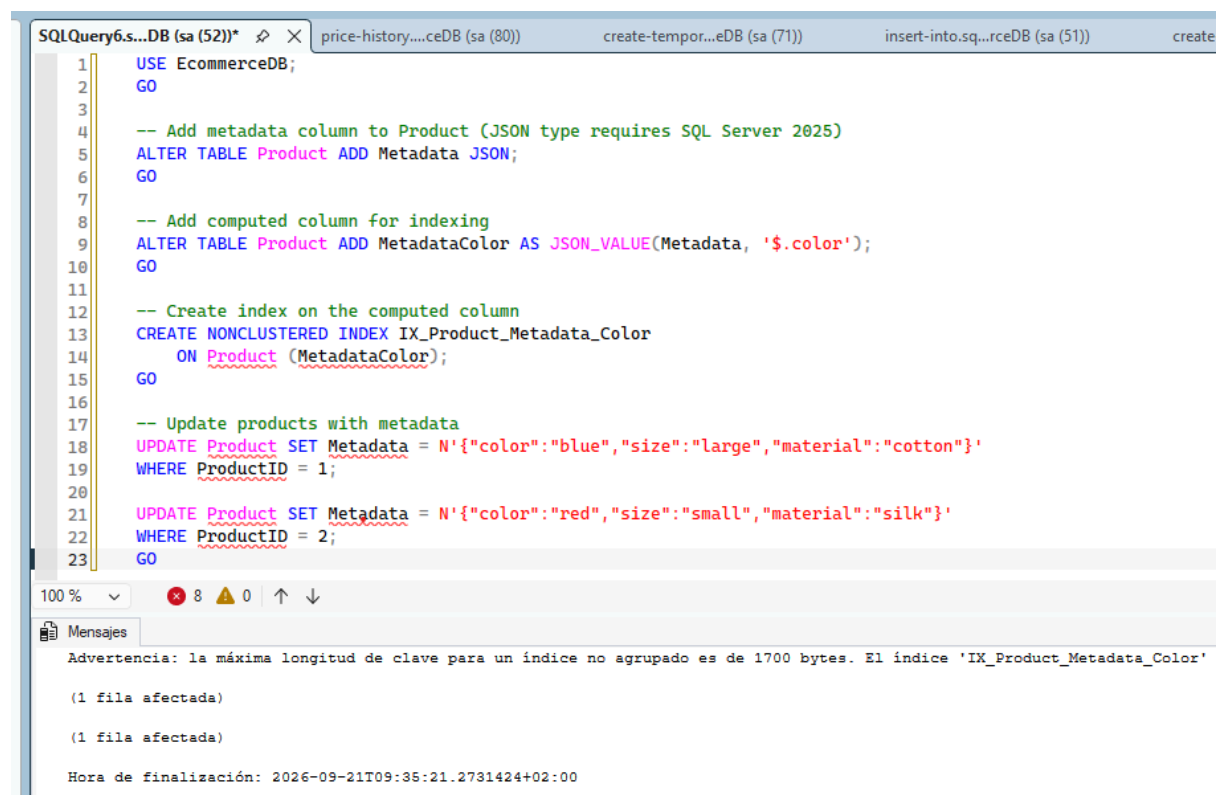
Below the query, the results are displayed in a table:

	ProductID	CurrentPrice	SysStartTime	SysEndTime
1	1	109.99	2026-09-21 07:25:30.4514674	9999-12-31 23:59:59.9999999
2	1	99.99	2026-09-21 07:25:30.4504406	2026-09-21 07:25:30.4514674

• Add JSON columns for metadata

- Añadimos una columna **JSON** a la tabla de productos para guardar metadatos y creamos un índice para consultar rápido por color.

1. Agregar columna JSON, columna calculada e índice.



The screenshot shows a SQL query window with the following code:

```
1 USE EcommerceDB;
2 GO
3
4 -- Add metadata column to Product (JSON type requires SQL Server 2025)
5 ALTER TABLE Product ADD Metadata JSON;
6 GO
7
8 -- Add computed column for indexing
9 ALTER TABLE Product ADD MetadataColor AS JSON_VALUE(Metadata, '$.color');
10 GO
11
12 -- Create index on the computed column
13 CREATE NONCLUSTERED INDEX IX_Product_Metadata_Color
14 ON Product (MetadataColor);
15 GO
16
17 -- Update products with metadata
18 UPDATE Product SET Metadata = N'{"color":"blue","size":"large","material":"cotton"}'
19 WHERE ProductID = 1;
20
21 UPDATE Product SET Metadata = N'{"color":"red","size":"small","material":"silk"}'
22 WHERE ProductID = 2;
23 GO
```

Below the query, the results are displayed in a table:

	ProductID	CurrentPrice	SysStartTime	SysEndTime
1	1	109.99	2026-09-21 07:25:30.4514674	9999-12-31 23:59:59.9999999
2	1	99.99	2026-09-21 07:25:30.4504406	2026-09-21 07:25:30.4514674

Below the table, the messages are displayed:

Advertencia: la máxima longitud de clave para un índice no agrupado es de 1700 bytes. El índice 'IX_Product_Metadata_Color'

(1 fila afectada)

(1 fila afectada)

Hora de finalización: 2026-09-21T09:35:21.2731424+02:00

2. Consultar los datos en formato JSON.

The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a SQL query with the following code:

```
1  USE EcommerceDB;
2  GO
3
4  -- Consultar y extraer atributos específicos del JSON
5  SELECT
6      ProductID,
7      ProductName,
8      JSON_VALUE(Metadata, '$.color') AS Color,
9      JSON_VALUE(Metadata, '$.size') AS Size,
10     JSON_VALUE(Metadata, '$.material') AS Material
11  FROM Product
12  WHERE JSON_VALUE(Metadata, '$.color') = 'blue';
13  GO
```

The bottom pane shows the execution results. A message box indicates an error: "Nombre de columna 'Metadata' no válido." (Column name 'Metadata' is not valid). Below the message, the results are displayed in a table:

	ProductID	ProductName	Color	Size	Material
1	1	Wireless Mouse	blue	large	cotton

• Create a partitioned order table

- Dividimos la tabla **Order** en particiones por fecha para gestionar grandes volúmenes de pedidos eficientemente.

1. Crear función, esquema, tabla e índice particionado.

```
SQLQuery8.s...DB (sa (54))*  query-JSN.sq...eDB (sa (72))  add-json.sql -...ceDB (sa (52))  price-history....ceDB (sa (53))
1  USE EcommerceDB;
2  GO
3
4  -- Create partition function for order dates
5  -- Use RANGE RIGHT for date columns to keep same-day values together
6  CREATE PARTITION FUNCTION PF_OrderDate (DATE)
7  AS RANGE RIGHT FOR VALUES
8  ('2025-01-01', '2025-04-01', '2025-07-01', '2025-10-01');
9
10 -- Create partition scheme (single filegroup recommended)
11 CREATE PARTITION SCHEME PS_OrderDate
12 AS PARTITION PF_OrderDate ALL TO ([PRIMARY]);
13
14 -- Create partitioned Order table
15 -- Include OrderDate in primary key for clustered index alignment
16 CREATE TABLE [Order] (
17     OrderID BIGINT IDENTITY(1,1),
18     OrderDate DATE NOT NULL,
19     CustomerName NVARCHAR(100) NOT NULL,
20     TotalAmount DECIMAL(12,2) NOT NULL,
21     OrderStatus NVARCHAR(20) DEFAULT 'Pending',
22     CONSTRAINT PK_Order PRIMARY KEY (OrderID, OrderDate),
23     CHECK (TotalAmount > 0),
24     CHECK (OrderStatus IN ('Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled'))
25 ) ON PS_OrderDate(OrderDate);
26
27 -- Create partitioned index
28 CREATE NONCLUSTERED INDEX IX_Order_Customer
29 ON [Order](CustomerName)
30 ON PS_OrderDate(OrderDate);
31 GO
32
33 -- Insert sample orders
34 INSERT INTO [Order] (OrderDate, CustomerName, TotalAmount, OrderStatus) VALUES
35 ('2025-01-15', 'John Smith', 299.97, 'Delivered'),
36 ('2025-02-20', 'Jane Doe', 149.99, 'Shipped'),
37 ('2025-06-10', 'Bob Johnson', 449.95, 'Processing');
38 GO
```

100 % No se encontraron problemas.

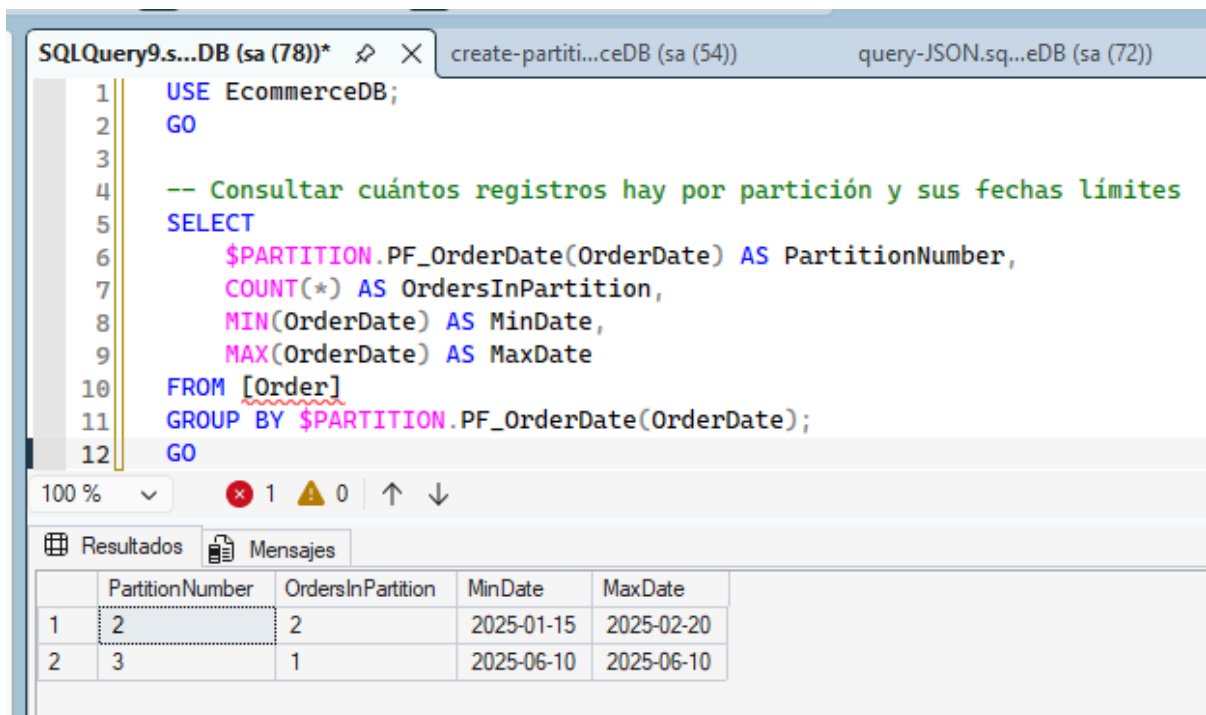
Mensajes

Se ha creado correctamente el esquema de partición 'PS_OrderDate'. 'PRIMARY' tiene la marca de grupo de a:

(3 filas afectadas)

Hora de finalización: 2026-09-21T09:40:02.0698384+02:00

2. Consultar la distribución de particiones.



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a query window with the following SQL code:

```
1 USE EcommerceDB;
2 GO
3
4 -- Consultar cuántos registros hay por partición y sus fechas límites
5 SELECT
6     $PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber,
7     COUNT(*) AS OrdersInPartition,
8     MIN(OrderDate) AS MinDate,
9     MAX(OrderDate) AS MaxDate
10  FROM [Order]
11  GROUP BY $PARTITION.PF_OrderDate(OrderDate);
12  GO
```

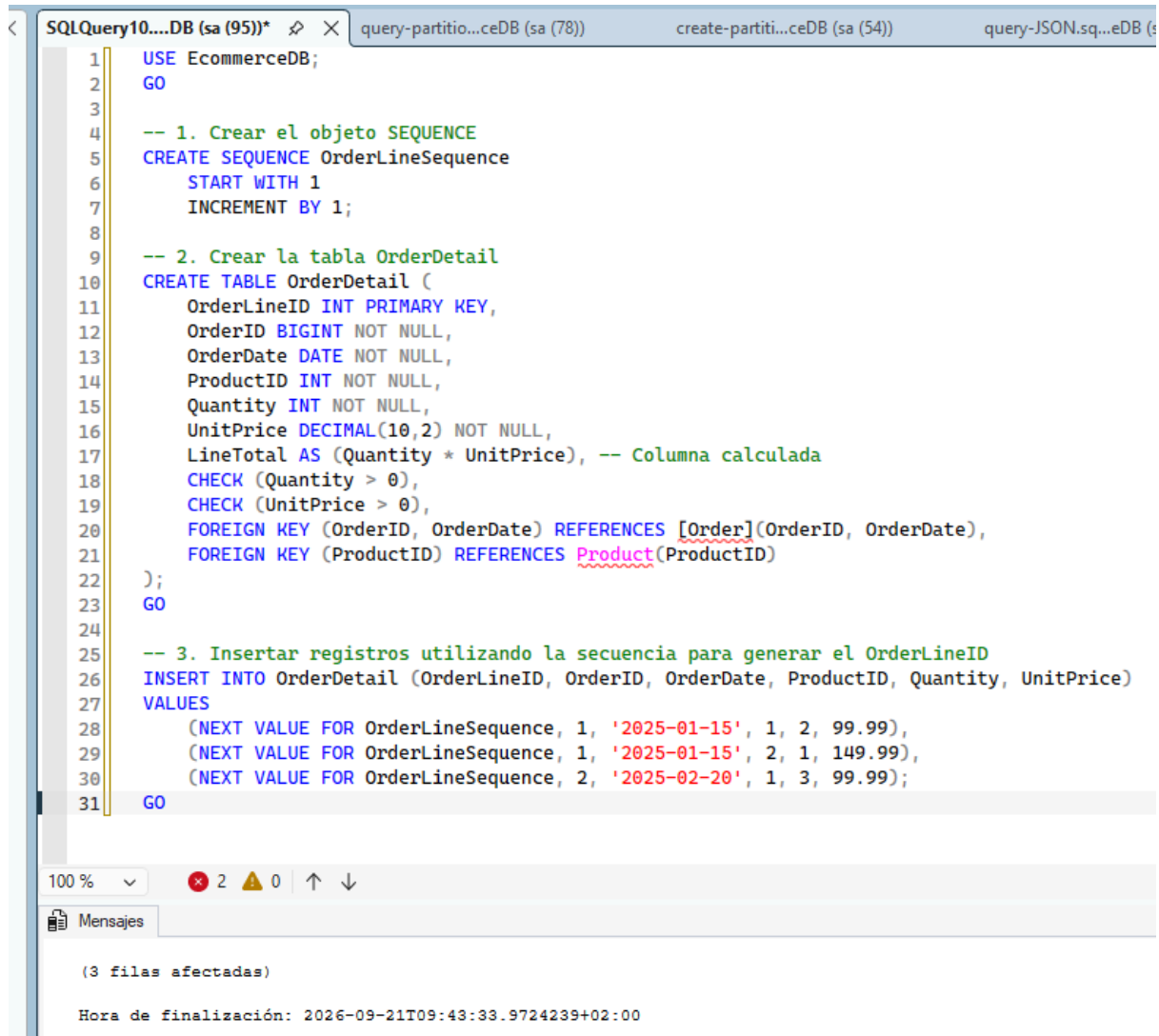
Below the query window, the 'Resultados' (Results) tab is active, showing a table with the following data:

	PartitionNumber	OrdersInPartition	MinDate	MaxDate
1	2	2	2025-01-15	2025-02-20
2	3	1	2025-06-10	2025-06-10

• Create order details with SEQUENCE

- Usamos un objeto **SEQUENCE** para generar números correlativos únicos en la tabla OrderDetail.

1. Crear la secuencia, la tabla e insertar datos.



```
1  USE EcommerceDB;
2  GO
3
4  -- 1. Crear el objeto SEQUENCE
5  CREATE SEQUENCE OrderLineSequence
6      START WITH 1
7      INCREMENT BY 1;
8
9  -- 2. Crear la tabla OrderDetail
10 CREATE TABLE OrderDetail (
11     OrderLineID INT PRIMARY KEY,
12     OrderID BIGINT NOT NULL,
13     OrderDate DATE NOT NULL,
14     ProductID INT NOT NULL,
15     Quantity INT NOT NULL,
16     UnitPrice DECIMAL(10,2) NOT NULL,
17     LineTotal AS (Quantity * UnitPrice), -- Columna calculada
18     CHECK (Quantity > 0),
19     CHECK (UnitPrice > 0),
20     FOREIGN KEY (OrderID, OrderDate) REFERENCES [Order](OrderID, OrderDate),
21     FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
22 );
23 GO
24
25 -- 3. Insertar registros utilizando la secuencia para generar el OrderLineID
26 INSERT INTO OrderDetail (OrderLineID, OrderID, OrderDate, ProductID, Quantity, UnitPrice)
27 VALUES
28     (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 1, 2, 99.99),
29     (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 2, 1, 149.99),
30     (NEXT VALUE FOR OrderLineSequence, 2, '2025-02-20', 1, 3, 99.99);
31 GO
```

100 % 2 0

Mensajes

(3 filas afectadas)

Hora de finalización: 2026-09-21T09:43:33.9724239+02:00

2. Verificar los datos insertados.

The screenshot shows a SQL query window titled 'SQLQuery11....DB (sa (75))'. The query is as follows:

```

1  USE EcommerceDB;
2  GO
3
4  SELECT * FROM OrderDetail;

```

The results are displayed in a table with the following columns: OrderLineID, OrderID, OrderDate, ProductID, Quantity, UnitPrice, and LineTotal.

	OrderLineID	OrderID	OrderDate	ProductID	Quantity	UnitPrice	LineTotal
1	1	1	2025-01-15	1	2	99.99	199.98
2	2	1	2025-01-15	2	1	149.99	149.99
3	3	2	2025-02-20	1	3	99.99	299.97

• Verify database objects

- Validamos el correcto funcionamiento de las reglas, consultas JSON, particiones e historial temporal.
1. **Verificar la restricción CHECK:** Ejecutaremos una inserción con un precio negativo. Esta consulta debe fallar a propósito para confirmar que la regla de validación funciona.

The screenshot shows a SQL query window titled 'SQLQuery12....DB (sa (88))'. The query is as follows:

```

1  USE EcommerceDB;
2  GO
3
4  -- Verify constraints work
5  -- This should fail: negative price
6  INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
7  VALUES ('Invalid', 1, 1, -50, 10);

```

The results are displayed in a table with the following columns: Mensaje, Nivel, Estado, and Línea.

Mensaje	Nivel	Estado	Línea
Mens. 547, Instrucción INSERT en conflicto con la restricción CHECK 'CK_Product__BasePri_534D60F1'. El conflicto ha aparecido en la base de datos 'EcommerceDB', tabla 'dbo.Product', column 'BasePrice'. Se terminó la instrucción.	16	0	6

La hora de finalización es: 2026-09-21T09:46:43.0308680+02:00.

2. Comprobar JSON, particionamiento e historial temporal.

SQLQuery13....DB (sa (53))* verification-q...ceDB (sa (88)) verify-data.sq...ceDB (sa (75)) create-seq

```
1 USE EcommerceDB;
2 GO
3
4 -- 1. Verificar consultas JSON
5 SELECT ProductName, JSON_VALUE(Metadata, '$.color') AS Color
6 FROM Product
7 WHERE Metadata IS NOT NULL;
8
9 -- 2. Verificar particionamiento de pedidos
10 SELECT $PARTITION.PF_OrderDate(OrderDate) AS Partition, COUNT(*) AS RecordCount
11 FROM [Order]
12 GROUP BY $PARTITION.PF_OrderDate(OrderDate);
13
14 -- 3. Verificar historial completo en la tabla temporal
15 SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
16 FROM ProductPrice FOR SYSTEM_TIME ALL
17 ORDER BY ProductID, SysStartTime;
18 GO
```

100 % 12 0

Resultados Mensajes

	ProductName	Color
1	Wireless Mouse	blue
2	Cotton T-Shirt	red

	Partition	RecordCount
1	2	2
2	3	1

	ProductID	CurrentPrice	SysStartTime	SysEndTime
1	1	99.99	2026-09-21 07:25:30.4504406	2026-09-21 07:25:30.4514674
2	1	109.99	2026-09-21 07:25:30.4514674	9999-12-31 23:59:59.9999999
3	2	149.99	2026-09-21 07:25:30.4504406	9999-12-31 23:59:59.9999999

• Cleanup

SQLQuery14.s...er (sa (77))* verification-J...ceDB (sa (53)) verification-q...ceDB (sa (88)) verify-data.sq...ceDB (sa (75))

```
1 USE master;
2 GO
3
4 ALTER DATABASE EcommerceDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
5 DROP DATABASE EcommerceDB;
6 GO
```

base de datos EcommerceDB

100 % No se encontraron problemas.

Mensajes

Se están revirtiendo las transacciones no calificadas. Estimación de porcentaje de reversión: 0%.

Se están revirtiendo las transacciones no calificadas. Estimación de porcentaje de reversión: 100%.

Hora de finalización: 2026-09-21T09:51:44.6717519+02:00